

Local LLM PDF Question Answering API Demo

Purpose of this PDF

This sample document is designed for a classroom demonstration of a local Large Language Model service deployed with Docker, FastAPI, and Uvicorn. The client application uploads this PDF along with a question. The API extracts text from the PDF, sends the text and question to a locally downloaded language model, and returns an answer as JSON.

The demonstration shows an industry-style pattern where a client program does not directly load the model. Instead, it calls a model API through a URL and sends data as a payload. The server owns the model, performs inference, and sends the response back to the client.

Key Learning Goals

- Understand how Docker packages an AI service with Python, libraries, and model-serving code.
- Understand how FastAPI creates endpoints such as `/load-model` and `/ask-pdf`.
- Understand how Uvicorn runs the FastAPI application as an HTTP server.
- Understand how a client program sends PDF data and a question to a model API.
- Understand how a local LLM can be downloaded and used for offline-style inference.

Demo Architecture

Layer	Role
client.py	Reads the PDF, sends the PDF and question to the API, receives JSON response, and performs inference.
FastAPI	Defines API endpoints and connects each URL to a Python function.
Uvicorn	Runs the web server and listens for HTTP requests on port 8000.
Local LLM	Generates an answer from the extracted PDF text and question.
Docker Container	Provides isolated runtime containing Python, dependencies, API code, and model execution environment.

How the Request Flows

When the user runs `client.py`, the client program first calls the `/load-model` endpoint. This loads the tokenizer and local language model inside the API server. Next, the client calls `/ask-pdf` and sends two things: the PDF file and the question. The API receives the file, stores it temporarily, extracts text using PyMuPDF, builds a prompt, sends the prompt to the local model, and returns the generated answer.

Step-by-Step Execution

- Step 1: Docker starts the container from the image.
- Step 2: Uvicorn starts automatically using the CMD instruction in the Dockerfile.
- Step 3: Uvicorn exposes the FastAPI application on port 8000.
- Step 4: The client sends an HTTP POST request to `/load-model`.
- Step 5: The API downloads or loads the selected local LLM.
- Step 6: The client uploads a PDF and sends a question to `/ask-pdf`.
- Step 7: The server extracts the PDF content and gives it to the LLM.
- Step 8: The model generates an answer and FastAPI returns it as JSON.

Important Concept: Where Does the Model Run?

In this demo, the local LLM runs inside the Docker container if the API server is started using `docker run`. Codespaces provides the host machine and Docker engine, but the Python process, FastAPI application, Uvicorn server, PDF extraction, and LLM inference run inside the container.

The model files are usually cached inside the container environment unless a Docker volume is mounted. If the container is removed, downloaded model files and generated temporary files may also be removed. For persistent storage, a host directory should be mounted into the container.

Example JSON Response

The `/ask-pdf` endpoint returns a response similar to the following:

```
{
  "question": "What is the main topic of this PDF?",
  "answer": "The PDF explains how to build a local LLM based PDF question answering API using Docker, FastAPI, Uvicorn, and a Python client.",
  "pdf_file": "sample.pdf",
  "characters_used_from_pdf": 3000
}
```

Component Responsibilities

Each component in the system has a separate responsibility. Separating responsibilities makes the architecture easy to test, deploy, and scale.

Component	Responsibility
Docker	Packages the application, dependencies, and runtime environment into a portable container.
Dockerfile	Acts as the recipe used to build the Docker image.
Docker Image	A packaged template that contains the code and required dependencies.
Docker Container	A running instance of the image where the actual application executes.
Uvicorn	Accepts HTTP requests and forwards them to the FastAPI application.
FastAPI	Maps URLs to Python functions and returns JSON responses.
Transformers Library	Loads tokenizer and model and performs text generation.
PyMuPDF	Extracts readable text from PDF pages.
client.py	Acts as an external application that calls the API using URL, file upload, and payload.

Classroom Analogy

Think of the architecture like a bank. Docker is the bank building that provides an isolated place to operate. Uvicorn is the security desk that receives visitors at the entrance. FastAPI is the receptionist who checks which service the visitor needs and sends the request to the correct department. The local LLM is the expert officer who reads the document and answers the question. The client program is the customer who submits the document and asks for information.

Why Use an API Instead of Direct Model Calls?

- The model remains centralized on the server.
- Many clients can call the same model API.
- Clients do not need to install large ML libraries.
- The model can be updated without changing the client code.
- The API can be secured, logged, monitored, and scaled.

Practical Use Cases

The same pattern can be applied to many real-world applications. A legal assistant can upload contracts and ask clause-related questions. A student assistant can upload lecture notes and ask for summaries. A customer support system can upload manuals and ask troubleshooting questions. A research assistant can upload papers and ask for contributions, methods, limitations, and datasets.

Example Questions to Ask This PDF

- What is the main topic of this PDF?
- What is the role of Docker in this architecture?
- What does Uvicorn do?
- What does FastAPI do?
- Where does the local LLM run when Docker is used?
- Why is client-server architecture useful for AI applications?

Limitations of the Simple Demo

This demo intentionally keeps the implementation simple. It extracts only the first part of the PDF text to fit into the context window of a small model. For production systems, the PDF should be chunked into smaller passages, embedded into a vector database, retrieved based on the question, and then passed to the LLM. That improved approach is called Retrieval-Augmented Generation, or RAG.

Small local models may produce short or imperfect answers. They are useful for classroom demonstrations because they are easier to download and run on CPU. Larger models usually give better answers but require more memory and compute.

Production Improvements

- Use vector search to retrieve only relevant PDF chunks.
- Store uploaded documents using document IDs.
- Add authentication for API access.
- Add logging and request tracing.
- Add a database to store questions and answers.
- Use GPU acceleration for faster inference.
- Mount Docker volumes to persist downloaded models and uploaded PDFs.

Final Summary

This PDF demonstrates a practical architecture for local LLM based document question answering. The client program calls an API through a URL, sends a PDF and question as payload, and receives an answer as JSON. The FastAPI server handles routing, Uvicorn runs the HTTP server, Docker provides the isolated execution environment, PyMuPDF extracts text, and the local LLM generates the answer.

The key teaching point is separation of roles. The client does not directly run the model. The API server owns the model and exposes it through endpoints. This separation is the foundation of AI microservices and real-world model deployment.

Quick Revision Points

- FastAPI creates APIs such as /load-model and /ask-pdf.
- Uvicorn runs FastAPI as a web server on port 8000.
- Docker container runs the complete application in isolation.
- The local LLM is downloaded and used by the server.
- The client sends PDF and question using HTTP POST.
- The server returns a JSON answer.

One-Line Explanation

This project shows how a Python client can call a Dockerized FastAPI service that uses a locally downloaded LLM to answer questions from a PDF document.